

Q1. Create a pure function in Java using Function.

A pure function always produces the same output for the same input and causes no observable side effects (it does not modify any external or shared state).

```
import java.util.function.Function;

public class PureFunctionDemo {
    public static void main(String[] args) {
        Function<Integer, Integer> doubleValue = x -> x * 2;

        System.out.println(doubleValue.apply(5)); // always 10
        System.out.println(doubleValue.apply(5)); // always 10 again
    }
}
```

The lambda `x -> x * 2`, assigned to the functional interface `Function`, depends only on its input `x` and produces a new result without modifying any external variable, making it a pure function.

Q2. What is a higher-order function? Give a Java example.

A higher-order function is a function that either takes one or more functions as parameters, returns a function as its result, or both. Java supports this through functional interfaces such as `Function`, `Consumer`, and `Supplier`.

```
import java.util.function.Function;

public class HigherOrderDemo {
    // Higher-order function: accepts a Function as a parameter
    static int applyTwice(Function<Integer, Integer> f, int value) {
        return f.apply(f.apply(value));
    }

    public static void main(String[] args) {
        Function<Integer, Integer> addFive = x -> x + 5;
        System.out.println(applyTwice(addFive, 10)); // 10 + 5 + 5 = 20
    }
}
```

The method `applyTwice` is a higher-order function because it accepts another function, `f`, as a parameter and applies it internally, rather than performing a fixed, hard-coded

operation.

Q3. Show immutability using final variables in Java.

Declaring a variable final prevents it from being reassigned once it has been initialized, which supports immutability.

```
public class ImmutabilityDemo {  
    public static void main(String[] args) {  
        final int MAX_USERS = 100;  
        System.out.println("Max users allowed: " + MAX_USERS);  
  
        // MAX_USERS = 200; // Compile-time error: cannot assign a value to  
        // a final variable  
    }  
}
```

Once MAX_USERS is assigned the value 100, the final keyword guarantees it cannot be changed anywhere else in the program; any attempt to reassign it results in a compile-time error. This is commonly used to define constants and to support the functional-programming principle of avoiding mutable state, since final local variables are also required when they are referenced inside a lambda expression or an anonymous class.